

Safenet Utils

Contracts

ENGAGEMENT	SECURITY REVIEW
REVISION	1.1
DELIVERED	2026-10-02
LEAD AUDITOR	Naoki Yoshida

Contents

1. Document Revisions.....	3
2. Overview.....	4
2.1. ack3.....	4
2.2. Review Methodology.....	5
2.3. Finding Classification	6
2.4. Review Team	8
2.5. Disclaimer	8
3. Executive Summary.....	9
Revision 1.0.....	9
Revision 1.1	10
4. Findings Summary.....	11
Report Revision 1.0.....	12
Revision Team	12
System Overview.....	12
Trust Model	12
Findings.....	12
Report Revision 1.1	17
Revision Team	17
System Overview.....	17
Trust Model.....	17
Appendix A: How to cite	18

1. Document Revisions

1.0-draft	Draft Report	September 25, 2026
1.1	Final Report	October 2, 2026

2. Overview

This document presents the findings of our review.

2.1. ack3

[ack3](#) is an in-house team of security researchers performing audits focused on manual code reviews with extensive fuzz testing and AI-assisted workflows for Ethereum and Solana. The ack3 team is trusted by top-tier organizations in web3, including Lido, Safe, and Axelar.

We develop the open-source security and developer tooling [Wake](#), which powers the fuzzing, static analysis, and graph-based review on our audits and helps auditors discover hardly recognizable edge-case vulnerabilities.

Our team teaches blockchain security at the [Czech Technical University in Prague](#), led by founder Josef Gattermayer, Ph.D.

For ack3, security is a long-term partnership: we work alongside our clients from design through post-deployment.

2.2. Review Methodology

We begin each review by mapping the codebase and the interactions between contracts and external components using [Wake](#)'s graph tooling. We work with the provided documentation and materials to discover discrepancies between the expected behavior and the actual implementation. Where applicable, we build a mathematical model of the protocol to cross-compare with the code and identify subtle bugs and inconsistencies.

At the start of the review, we launch an automated [ack3](#) AI scan pinned to the scoped commits, running in parallel with the manual review. The scan provides additional coverage and new leads for the team to analyze; each scan finding is manually reviewed by the team.

During the review, we apply AI-assisted workflows and skills that support codebase onboarding and navigation, combined with Wake's static analyzer and language server for context-aware code analysis.

We launch AI agents to build on our vulnerability hypotheses and expand the attack surface with new escalation paths. All findings are manually reviewed and validated by the team. Where applicable, we confirm findings and verify the exact behavior of the code with unit tests and fuzzing. Where the engagement includes a fuzzing campaign, it is manually guided by an auditor; the fuzz tests themselves are AI-generated.

Once sufficient context is gathered, we search for deeper, chained issues based on the attack vectors discovered during review. This includes combining smaller weaknesses, testing trust-boundary assumptions, and evaluating whether unexpected behavior can lead to practical impact.

The validated results are documented in the report: findings with severity, affected components, impact, and recommended remediation.

2.3. Finding Classification

The Severity of each finding combines two sub-ratings, Impact and Likelihood, and ranges from Info to Critical.

If we find a scenario in which an issue is exploitable, it is assigned an impact rating of High, Medium, or Low, based on its consequences for the system. If we find no way to exploit it, or the issue is only exploitable given a change in configuration (system settings or parameters, such as deployment scripts, compiler configurations, or multi-signature wallets for owners) or in the codebase, it is assigned Warning or Info.

Low to High impact issues also have a Likelihood, which measures the probability of exploitation at runtime.

Severity

		LIKELIHOOD			
		HIGH	MEDIUM	LOW	N/A
IMPACT	HIGH	Critical	High	Medium	-
	MEDIUM	High	Medium	Low	-
	LOW	Medium	Low	Low	-
	WARNING	-	-	-	Warning
	INFO	-	-	-	Info

Table 1. Severity of findings

Impact

- **High** - Code that activates the issue leads to undefined or catastrophic consequences for the system.
- **Medium** - Code that activates the issue results in serious consequences.
- **Low** - Code that activates the issue has consequences that are recoverable or do not jeopardize the system's normal operation.
- **Warning** - The issue cannot be exploited given the current code or configuration, but could become a security vulnerability if either changed. If we have not found a way to exploit the issue given the time constraints, it may be marked as Warning or higher, based on our best estimate of whether it is currently exploitable.
- **Info** - The issue is on the borderline between code quality and security. Examples include insufficient logging for critical operations, or issues that would become security-related if code or configuration changed.

Likelihood

- **High** - The issue is exploitable by virtually anyone under any circumstance.
- **Medium** - Exploiting the issue requires non-trivial preconditions.
- **Low** - Exploiting the issue requires strict preconditions.

2.4. Review Team

For a revision’s authors, see the “Revision team” section in that revision’s chapter.

NAME	POSITION
Naoki Yoshida	Lead Auditor
Josef Gattermayer, Ph.D.	Audit Supervisor

2.5. Disclaimer

We have made our best effort to find all vulnerabilities in the system; however, our findings should not be considered a complete list of issues. Statements in this document should not be interpreted as investment or legal advice, nor should the authors be held accountable for decisions based on them.

3. Executive Summary

Safenet Utils includes `RealityVetoModule`, which allows a designated vetoer to invalidate a proposal without granting broader control over the Safe.

Revision 1.0

Safe engaged [ack3](#) to perform a security review of Safenet Utils Contracts with a total donation of 1 engineering days between September 24 and September 25, 2026, with Naoki Yoshida as the lead reviewer.

The review was performed on the commit [b591c35^{\[1\]}](#) in the [contracts](#) repository and the scope was the following:

- `governance/contracts/src/RealityVetoModule.sol`.

We began our review using static analysis tools, including [Wake](#). We then examined the logic of the contracts in detail. For testing and writing exploit scenarios, we used the [Wake](#) testing framework. During the review, we paid special attention to:

- the permission structure;
- the unexpected function call;
- potential reentrancy in the code;
- overly permissive or overly restrictive access controls; and
- common issues, including problems with data validation.

Our review resulted in 2 informational findings.

[ack3](#) recommended that Safe take the following action:

- read and review the complete report.

See [Report Revision 1.0](#) for the system overview and trust model.

Revision 1.1

Safe engaged [ack3](#) to perform a fix review of the findings from the previous revision.

Safe provided fixes for the reported findings.

The review was performed between October 1 and October 1, 2026 on the commit [030bb80](#)^[2].

From the reported 2 findings:

- 2 issues were fixed;
- 0 issues were acknowledged; and
- 0 minor issues remained unresolved.

No new findings were reported.

See [Report Revision 1.1](#) for more details.

[1] full commit hash: b591c35b59c189b6462e5217cc3386ca992601d7, link to [commit](#)

[2] full commit hash: 030bb801ea3593a4b04e0e7ec133edbdd8b16f3a, link to [commit](#)

4. Findings Summary



Figure 1. Findings by Severity

FINDING TITLE	SEVERITY	REVISION	STATUS
11 : Incomplete module guard compatibility documentation	INFO	1.0	Fixed
12 : Incorrect veto question hash recipe	INFO	1.0	Fixed

Table 2. Table of Findings

Report Revision 1.0

Revision Team

NAME	POSITION
Naoki Yoshida	Lead Auditor
Josef Gattermayer, Ph.D.	Audit Supervisor

System Overview

The RealityVetoModule allows the vetoer to make the Safe call `markProposalAsInvalidByHash`. Without this module, the DAO must call the function through the Safe.

This gives the vetoer a way to reject a proposal in an emergency.

Trust Model

DAO users must trust the vetoer, who can mark any question hash as `INVALIDATED`.

Findings

This section lists the findings from this revision; for all findings, see the [Findings Summary](#).

I1: Incomplete module guard compatibility documentation

Impact:	Info	Likelihood:	N/A
Target:	RealityVetoModule.sol, README.md		

Description

The RealityVetoModule documentation states that module execution bypasses the Safe transaction guard and that the module's own access control is the entire security boundary.

Listing 1. Excerpt from [RealityVetoModule NatSpec](#)

```
11 * @dev Stateless by design: `SAFE`, `REALITY_MODULE` and `VETOER` are set
    once in the constructor and never
12 *      change, and `vetoProposal` performs no check beyond the caller. A
    module bypasses owner signatures,
13 *      the threshold and the transaction guard, so this contract's own
    access control is the whole security
14 *      boundary.
```

This is true for Safe 1.3.0, used in the tests, and Safe 1.4.1, documented for production. Safe 1.5.0 introduced module guards, which apply to `execTransactionFromModule`. If a configured module guard reverts, `vetoProposal` reverts, and the proposal is not invalidated.

Recommendation

If the Safe may be upgraded to 1.5.0 or later, document in the NatSpec that a configured module guard can block `vetoProposal`.

Fix 1.1

The finding was fixed by documenting that a configured module guard could block `vetoProposal`.

[Go back to Findings Summary](#)

I2: Incorrect veto question hash recipe

Impact:	Info	Likelihood:	N/A
Target:	README.md		

Description

The operator runbook derives `questionHash` from Cast's displayed string output instead of the raw bytes returned by `buildQuestion`. `cast call ... (string)` prints the decoded string with literal double quotes, so `cast keccak "$QUESTION"` hashes "`<question>`" instead of computing `keccak256(bytes(buildQuestion(...)))`. For `0x`-prefixed proposal IDs, stripping the display quotes is still unsafe because `cast keccak` may parse the value as hex.

Listing 2. Excerpt from [README](#)

```
86 QUESTION=$(cast call $VETO_REALITY_MODULE_ADDRESS
    "buildQuestion(string,bytes32[])(string)" "$PROPOSAL_ID" "[TX_HASHES]"
    --rpc-url $RPC_URL)
87 QUESTION_HASH=$(cast keccak "$QUESTION")
```

Because `vetoProposal` forwards the supplied `bytes32`, the documented procedure can invalidate an unrelated hash and emit `ProposalVetoed` while the targeted proposal remains executable.

Recommendation

Derive the question hash from the exact UTF-8 bytes returned by `buildQuestion`. Update the runbook to call `buildQuestion` without a declared return type, ABI-decode the raw return data as bytes, and pass those hex bytes to `cast keccak`.

Fix 1.1

The finding was fixed by correcting the question hash calculation in the runbook.

[Go back to Findings Summary](#)

Report Revision 1.1

Revision Team

NAME	POSITION
Naoki Yoshida	Lead Auditor
Josef Gattermayer, Ph.D.	Audit Supervisor

System Overview

The system overview remained unchanged since the previous revision.

Trust Model

The trust model remained unchanged since the previous revision.

Appendix A: How to cite

Cite this document as:

[ack3](#), Security Review | Safenet Utils: Contracts, October 2, 2026.



Thank you.

COMPANY	ACK3 LTD
WEB	ack3.ai
X	@ack3ai
MAIL	hello@ack3.ai